

Prueba Técnica – API .NET Senior

Objetivo

Desarrollar una API REST en .NET para administrar reservaciones de salas corporativas.

La solución debe contemplar:

- CRUD de entidades
- reglas de negocio
- filtros
- ordenamiento
- paginación
- persistencia en base de datos
- manejo adecuado de errores

Tiempo estimado

3 horas

Tecnologías requeridas

- .NET 6, .NET 7 o .NET 8
- ASP.NET Core Web API

Persistencia:

- SQLite, SQL Server LocalDB o PostgreSQL

ORM:

- Entity Framework Core (preferente)

La arquitectura es libre.

Contexto del negocio

La empresa requiere un sistema para administrar reservaciones de salas de reuniones.

Cada sala puede tener múltiples reservaciones, pero no deben existir traslapes de horario entre reservaciones activas de la misma sala.

Modelos

Room

La entidad debe contener al menos los siguientes campos:

- Id (int)
 - Name (string)
 - Capacity (int)
 - IsActive (bool)
 - CreatedAt (DateTime)
-

Reservation

La entidad debe contener al menos los siguientes campos:

- Id (int)
- RoomId (int)
- ReservedBy (string)
- Purpose (string)
- StartTime (DateTime)
- EndTime (DateTime)
- Status (string)
- CreatedAt (DateTime)

Valores permitidos para Status:

- Pending
- Confirmed

- Cancelled
-

Requerimientos

CRUD de salas

Implementar endpoints para:

- Crear sala
 - Obtener todas las salas
 - Obtener sala por ID
 - Actualizar sala
 - Eliminar sala
-

CRUD de reservaciones

Implementar endpoints para:

- Crear reservación
 - Obtener todas las reservaciones
 - Obtener reservación por ID
 - Actualizar reservación
 - Cancelar reservación
-

Reglas de negocio

Validaciones de Room

- Name es obligatorio
 - Name máximo 100 caracteres
 - Capacity debe ser mayor a 0
-

Validaciones de Reservation

- ReservedBy es obligatorio
 - Purpose es obligatorio
 - StartTime debe ser mayor a la fecha/hora actual
 - EndTime debe ser mayor a StartTime
 - Status solo puede contener:
 - Pending
 - Confirmed
 - Cancelled
-

Validaciones de negocio

Traslape de reservaciones

No se debe permitir crear o actualizar una reservación si existe traslape de horario con otra reservación activa de la misma sala.

Ejemplo inválido:

Reserva 1:

- 10:00 AM - 11:00 AM

Reserva 2:

- 10:30 AM - 11:30 AM
-

Otras reglas

- Reservaciones con status Cancelled no deben bloquear horarios
 - No se puede reservar una sala inactiva
 - No se puede modificar una reservación cancelada
 - No se puede eliminar una sala que tenga reservaciones futuras activas
-

Filtros

El endpoint:

GET /api/reservations

Debe soportar filtros opcionales:

- roomId
- status
- reservedBy
- startDate
- endDate

Ejemplo:

GET /api/reservations?status=Confirmed&roomId=1&startDate=2026-05-01

Paginación

El endpoint GET debe soportar:

- page (default: 1)
- pageSize (default: 10, máximo: 50)

Respuesta esperada:

```
{  
  "data": [...],  
  "total": 120,  
  "page": 1,  
  "pageSize": 10  
}
```

Ordenamiento

Parámetros opcionales:

- sortBy
 - startTime
 - endTime

- createdAt
 - sortOrder
 - asc
 - desc
-

Persistencia

La información debe persistirse en base de datos.

Se espera:

- configuración funcional
 - migraciones necesarias
 - instrucciones básicas de ejecución
-

Manejo de errores

Se espera:

- Uso correcto de códigos HTTP
 - Validaciones claras
 - Respuestas consistentes
 - Manejo adecuado de recursos inexistentes
 - Manejo adecuado de conflictos de negocio
-

Extras opcionales

Si el tiempo lo permite, agregar cualquiera de los siguientes:

- Swagger documentado
- Dockerfile
- Autenticación JWT básica
- Logs estructurados

- Pruebas unitarias
-

Entregables

Enviar:

- Código fuente completo
 - Instrucciones de ejecución
 - Script/migraciones necesarias
 - Colección Postman o Swagger
-

Consideraciones

- La estructura del proyecto es libre
- Puede usar controllers, minimal APIs o el enfoque que prefiera
- Se evaluará claridad, simplicidad y buenas prácticas
- Se evaluará más la calidad de la solución que la cantidad de código